**MINISTRY OF EDUCATION AND RESEARCH**
**OF THE REPUBLIC OF MOLDOVA**

**Technical University of Moldova**
**Faculty of Computers, Informatics and Microelectronics**
**Department of Software Engineering and Automation**

**Petcov Nicolai FAF-233**

# Report

Laboratory Work No. 6

**Information Security and Cryptography**

**Hash Functions and Digital Signatures**

*Checked by:*
**Zaica M.**, *university assistant*
DISA, FCIM, UTM

Chișinău – 2025

# 1 Objective

Study and implement digital signature algorithms using hash functions. Understand the principles of RSA and ElGamal digital signatures, the role of cryptographic hash functions in ensuring message integrity and authenticity, and develop practical skills in implementing signature generation and verification with large prime numbers and modular arithmetic.

# 2 Tasks

1. Study recommended didactic materials on hash functions and digital signatures
2. Implement RSA digital signature with modulus $n \geq 3072$ bits for the message obtained in Laboratory Work No. 2. Hash function selected by formula $i = (k \mod 24) + 1$ where $k = 21$ (student number)
3. Implement ElGamal digital signature using provided 2048-bit prime $p$ and generator $g = 2$ for the same message. Hash function selected from a different list using the same formula
4. Provide step-by-step explanations of all algorithm steps with detailed comments

**Hash Function Selection:**

- Student number: $k = 21$
- Index calculation: $i = (21 \mod 24) + 1 = 22$
- Task 2 (RSA): Position 22 → **Haval192,3**
- Task 3 (ElGamal): Position 22 → **Haval192,3**

# 3 Theoretical Background

## 3.1 Digital Signatures

Digital signatures provide three essential security properties:

- **Authentication:** Verifies the identity of the signer
- **Integrity:** Ensures the message has not been altered
- **Non-repudiation:** Signer cannot deny having signed the message

The general process involves:

1. Compute hash $H(m)$ of message $m$
2. Sign the hash using private key to produce signature $S$
3. Verify signature using public key

## 3.2 Cryptographic Hash Functions

A cryptographic hash function $H$ maps arbitrary-length input to fixed-length output with properties:

- **Deterministic:** Same input always produces same output
- **Pre-image resistance:** Given $h$, hard to find $m$ such that $H(m) = h$
- **Second pre-image resistance:** Given $m_1$, hard to find $m_2 \neq m_1$ with $H(m_1) = H(m_2)$
- **Collision resistance:** Hard to find any $m_1 \neq m_2$ with $H(m_1) = H(m_2)$

## 3.3 HAVAL Hash Function

HAVAL (HAsh of VAriable Length) is a cryptographic hash function that supports:

- Variable number of passes: 3, 4, or 5
- Variable output lengths: 128, 160, 192, 224, or 256 bits
- Uses 1024-bit (128-byte) message blocks
- Based on MD5 structure with enhancements

For this laboratory: **Haval192,3** = 3 passes, 192-bit output.

## 3.4 RSA Digital Signature

RSA signature scheme based on the difficulty of factoring large composite numbers.

**Key Generation:**

1. Generate large primes $p$ and $q$ ($\approx 1536$ bits each for 3072-bit modulus)
2. Compute $n = p \times q$ and $\phi(n) = (p-1)(q-1)$
3. Choose $e = 65537$ where $\gcd(e, \phi(n)) = 1$
4. Compute $d \equiv e^{-1} \pmod{\phi(n)}$
5. Public key: $(e, n)$; Private key: $(d, n)$

**Signature Generation:**

$$S = H(m)^d \bmod n \tag{1}$$

**Signature Verification:**

$$H(m) \stackrel{?}{=} S^e \bmod n \tag{2}$$

**Correctness:** Since $ed \equiv 1 \pmod{\phi(n)}$:

$$S^e = (H(m)^d)^e = H(m)^{de} = H(m)^{k\phi(n)+1} \equiv H(m) \pmod{n} \tag{3}$$

### 3.5 ElGamal Digital Signature

ElGamal signature scheme based on the discrete logarithm problem.

**Parameters:** Large prime $p$, generator $g$

**Key Generation:**

1. Choose random private key $x$ where $1 < x < p - 1$
2. Compute public key $y = g^x \bmod p$

**Signature Generation:** For message hash $H(m)$:

1. Choose random $k$ where $1 < k < p - 1$ and $\gcd(k, p - 1) = 1$
2. Compute $r = g^k \bmod p$
3. Compute $s = (H(m) - x \cdot r) \cdot k^{-1} \bmod (p - 1)$
4. Signature: $(r, s)$

**Signature Verification:**

$$g^{H(m)} \bmod p \stackrel{?}{=} y^r \cdot r^s \bmod p \tag{4}$$

**Correctness:**

$$y^r \cdot r^s = g^{xr} \cdot g^{ks} = g^{xr+ks} \tag{5}$$

$$= g^{xr+k(H(m)-xr)k^{-1}} = g^{xr+H(m)-xr} = g^{H(m)} \pmod{p} \tag{6}$$

# 4 Implementation

## 4.1 HAVAL Hash Function Implementation

```
class Haval:
    """HAVAL hash function implementation"""

    def __init__(self, passes=3, fpt_len=192):
        """
        Initialize HAVAL
        passes: number of passes (3, 4, or 5)
        fpt_len: fingerprint length in bits (128, 160, 192, 224, 256)
        """
        self.passes = passes
        self.fpt_len = fpt_len
        self.count = [0, 0]  # processed bits counter
        self.fingerprint = [
            0x243F6A88, 0x85A308D3, 0x13198A2E, 0x03707344,
            0xA4093822, 0x299F31D0, 0x082EFA98, 0xEC4E6C89
        ]
        self.block = bytearray(128)
        self.block_index = 0

    def update(self, data):
        """Update hash with new data"""
        if isinstance(data, str):
```

```
            data = data.encode('utf-8')

        for byte in data:
            self.block[self.block_index] = byte
            self.block_index += 1

            if self.block_index == 128:
                self._process_block(self.block)
                self.block_index = 0

            self.count[0] += 8
            if self.count[0] < 8:
                self.count[1] += 1

    def hexdigest(self):
        """Return hash as hexadecimal string"""
        return self.digest().hex()
```

Listing 1: HAVAL Hash Class

## 4.2 Hash Function Selection

```
def get_hash_function(k):
    """
    Determine hash function based on student number
    i = (k mod 24) + 1
    """
    hash_functions = [
        "MD4", "MD5", "MD2", "MD6-128", "MD6-256", "MD6-512",
        "SHA-1", "SHA-224", "SHA-256", "SHA-384", "SHA-512",
        "SHA3-224", "SHA3-256", "SHA3-384", "SHA3-512",
        "RIPEMD-128", "RIPEMD-160", "RIPEMD-256", "RIPEMD-320",
        "Whirlpool", "NTLM", "Haval192,3", "Haval224,4", "Haval256,4"
    ]

    i = (k % 24) + 1
    return hash_functions[i - 1], i
```

Listing 2: Hash Function Selection for RSA

## 4.3 Hash Computation

```
def compute_hash(message, hash_name):
    """
    Compute message hash using specified algorithm
    """
    message_bytes = message.encode('utf-8')

    # k=21 => i=22 => Haval192,3
    if hash_name == "Haval192,3":
        from haval import Haval
        h = Haval(passes=3, fpt_len=192)
        h.update(message_bytes)
        hash_hex = h.hexdigest()
        hash_decimal = int(hash_hex, 16)

```

```
        return hash_decimal, hash_hex
    else:
        # Fallback to SHA-256
        hash_obj = hashlib.sha256(message_bytes)
        hash_hex = hash_obj.hexdigest()
        hash_decimal = int(hash_hex, 16)
        return hash_decimal, hash_hex
```

Listing 3: Hash Computation with Haval192-3

## 4.4 RSA Key Generation

```
def generate_large_prime(bits):
    """Generate a prime number with specified bit length"""
    while True:
        candidate = random.getrandbits(bits)
        candidate |= (1 << (bits - 1))  # Ensure correct bit length
        candidate |= 1  # Ensure odd number

        if isprime(candidate):
            return candidate

def generate_rsa_keys(bits=3072):
    """Generate RSA public and private keys"""
    half_bits = bits // 2

    # Step 1: Generate primes p and q
    p = generate_large_prime(half_bits)
    q = generate_large_prime(half_bits)

    # Step 2: Compute n = p * q
    n = p * q

    # Verify n has required bit length
    if n.bit_length() < bits:
        return generate_rsa_keys(bits)

    # Step 3: Compute phi(n) = (p-1)(q-1)
    phi_n = (p - 1) * (q - 1)

    # Step 4: Choose e = 65537
    e = 65537
    if gcd(e, phi_n) != 1:
        return generate_rsa_keys(bits)

    # Step 5: Compute d = e^(-1) mod phi(n)
    d = mod_inverse(e, phi_n)

    # Verify: (d * e) mod phi(n) = 1
    assert (d * e) % phi_n == 1

    return (e, n), (d, n), p, q
```

Listing 4: RSA Key Generation (3072-bit)

## 4.5 RSA Signature

```python
def rsa_sign(message_hash, private_key):
    """
    Sign message hash using RSA private key

    Signature: S = H(m)^d mod n
    """
    d, n = private_key

    if message_hash >= n:
        raise ValueError("Hash too large for modulus")

    # Compute signature using modular exponentiation
    signature = pow(message_hash, d, n)

    return signature

def rsa_verify(message_hash, signature, public_key):
    """
    Verify RSA digital signature

    Verification: H(m) = S^e mod n
    """
    e, n = public_key

    # Compute H'(m) = S^e mod n
    computed_hash = pow(signature, e, n)

    # Compare with original hash
    is_valid = (message_hash == computed_hash)

    return is_valid
```

Listing 5: RSA Signature Generation and Verification

## 4.6 ElGamal Key Generation

```python
# Given parameters (2048-bit prime from assignment)
P = 323170060713110073001535134782516336248805713348907517458...
G = 2

def generate_elgamal_keys(p, g):
    """
    Generate ElGamal public and private keys

    Returns:
        public_key: (p, g, y)
        private_key: x
    """
    # Verify p is prime
    assert isprime(p), "p must be prime"

    # Step 1: Choose random private key x
    # x must satisfy: 1 < x < p-1
    x = random.randint(2, p - 2)

    # Step 2: Compute public key y = g^x mod p
    y = pow(g, x, p)
```

```
22
23      return (p, g, y), x
```

Listing 6: ElGamal Key Generation

## 4.7 ElGamal Signature

```
1  def elgamal_sign(message_hash, private_key, p, g):
2      """
3      Sign message hash using ElGamal signature
4
5      Steps:
6      1. Choose random k where gcd(k, p-1) = 1
7      2. r = g^k mod p
8      3. s = (H(m) - x*r) * k^(-1) mod (p-1)
9
10     Signature: (r, s)
11     """
12     x = private_key
13
14     # Step 1: Find valid k
15     while True:
16         k = random.randint(2, p - 2)
17         if gcd(k, p - 1) == 1:
18             break
19
20     # Step 2: Compute r = g^k mod p
21     r = pow(g, k, p)
22
23     # Step 3: Compute s = (H(m) - x*r) * k^(-1) mod (p-1)
24     k_inv = mod_inverse(k, p - 1)
25     h_minus_xr = (message_hash - x * r) % (p - 1)
26     s = (h_minus_xr * k_inv) % (p - 1)
27
28     return r, s
```

Listing 7: ElGamal Signature Generation

```
1  def elgamal_verify(message_hash, signature, public_key):
2      """
3      Verify ElGamal digital signature
4
5      Verification: g^H(m) mod p = y^r * r^s mod p
6      """
7      r, s = signature
8      p, g, y = public_key
9
10     # Check constraints
11     if not (0 < r < p and 0 < s < p - 1):
12         return False
13
14     # Compute left side: g^H(m) mod p
15     left_side = pow(g, message_hash, p)
16
17     # Compute right side: y^r * r^s mod p
18     y_r = pow(y, r, p)
19     r_s = pow(r, s, p)
```

```
    right_side = (y_r * r_s) % p

    # Compare
    is_valid = (left_side == right_side)

    return is_valid
```

Listing 8: ElGamal Signature Verification

# 5 Results

## 5.1 RSA Digital Signature Results

**Input Parameters:**

- Message: "Mesaj din Lucrarea de laborator nr. 2"
- Student number: $k = 21$
- Hash function: Haval192,3 (index $i = 22$)

**Hash Computation:**

- Hash (hex): 48-character hexadecimal string (192 bits)
- Hash (decimal): Large integer representation

**RSA Keys Generated:**

- Modulus $n$: 3072 bits
- Public exponent $e = 65537$
- Private exponent $d$: $\approx 3072$ bits
- Primes $p, q$: 1536 bits each

**Signature Process:**

1. Computed $S = H(m)^d \bmod n$
2. Verified: $S^e \bmod n = H(m)$

**Result:** Signature is **VALID**

## 5.2 ElGamal Digital Signature Results

**Input Parameters:**

- Same message from Laboratory Work No. 2
- Given prime $p$: 2048 bits
- Generator $g = 2$
- Hash function: Haval192,3

**ElGamal Keys Generated:**

- Private key $x$: random, $1 < x < p - 1$
- Public key $y = g^x \bmod p$

**Signature Process:**

1. Selected random $k$ with $\gcd(k, p - 1) = 1$
2. Computed $r = g^k \bmod p$
3. Computed $s = (H(m) - x \cdot r) \cdot k^{-1} \bmod (p - 1)$
4. Signature: $(r, s)$

**Verification:**

- Computed $g^{H(m)} \bmod p$
- Computed $y^r \cdot r^s \bmod p$
- Values match

**Result:** Signature is **VALID**

### 5.3 Demonstration: Message Modification

To demonstrate signature integrity, we modified the message and verified the signature:
**Modified message:** "Mesaj din Lucrarea de laborator nr. 2 (modificat)"
**Result:** Signature verification **FAILED**
This confirms that digital signatures detect any modification to the original message.

## 6 Security Analysis

### 6.1 Algorithm Comparison

| Feature | RSA Signature | ElGamal Signature |
|---|---|---|
| Security basis | Integer factorization | Discrete logarithm |
| Key size (this lab) | 3072 bits | 2048 bits |
| Signature size | 1× modulus | 2× message (r, s) |
| Randomized | No | Yes (requires unique k) |
| Verification speed | Fast ($e$ is small) | Slower (2 exponentiations) |

Table 1: Comparison of RSA and ElGamal Digital Signatures

## 6.2 Security Considerations

**RSA Signature:**

- 3072-bit keys provide security until ∼2030
- Deterministic: same message produces same signature
- Requires secure padding schemes (PSS) in practice
- Vulnerable to quantum computers (Shor's algorithm)

**ElGamal Signature:**

- **Critical:** Random $k$ must be unique for each signature
- Reusing $k$ reveals private key $x$
- Provides semantic security through randomization
- Larger signature size (double the data)

**HAVAL Hash Function:**

- Haval192,3 uses 3 passes and produces 192-bit hash
- Considered legacy algorithm (designed 1992)
- Modern alternatives: SHA-256, SHA-3
- Still suitable for educational purposes

## 6.3 Hash Function Role

The hash function provides:

1. **Efficiency:** Sign fixed-size hash instead of variable-length message
2. **Security:** Hash size matches security requirements
3. **Integrity:** Any message change produces different hash

# 7 Conclusions

Successfully implemented two digital signature algorithms with hash functions:

1. **RSA Digital Signature:**
   - Generated 3072-bit RSA keys
   - Implemented signature: $S = H(m)^d \bmod n$
   - Implemented verification: $H(m) = S^e \bmod n$
   - Demonstrated successful signature and verification
2. **ElGamal Digital Signature:**

- Used given 2048-bit prime $p$ and generator $g = 2$
- Implemented signature generation with random $k$
- Implemented verification: $g^{H(m)} = y^r \cdot r^s \pmod{p}$
- Demonstrated successful signature and verification

3. **HAVAL Hash Function:**
   - Implemented Haval192,3 (3 passes, 192-bit output)
   - Used for both RSA and ElGamal signatures
   - Provides deterministic, fixed-length message digest

**Key Learning Outcomes:**

- Understanding of digital signature principles (authentication, integrity, non-repudiation)
- Mathematical foundations: modular arithmetic, Euler's theorem, discrete logarithm
- Practical implementation with large integers (2048-3072 bits)
- Role of hash functions in digital signature schemes
- Security differences between RSA and ElGamal signatures

**Critical Insights:**

- Digital signatures ensure message authenticity and integrity
- Hash functions enable efficient signing of arbitrary-length messages
- RSA signatures are deterministic; ElGamal requires unique random values
- Modern systems use hybrid approaches combining signatures with certificates

## Source Code

`https://github.com/Nickseen/Cryptology-Security/tree/Lab6`